```
initialize V(s) arbitrarily

repeat for each episode:
    initialize S

    while S is not terminal:
        choose A according to fixed policy pi
        take action A
        observe R, S_next

        target = R + gamma * V(S_next)
        delta  = target - V(S)
        V(S)   = V(S) + alpha * delta

        S = S_next
```

$$V(St) \leftarrow V(St) + \alpha[Rt + \mathbf{1} + \gamma V(St + \mathbf{1}) - V(St)]$$

- TD(0) state-value
  - 为什么最后更新是 V(S)
    - 这行：
      V(S) = V(S) + alpha * delta
      就是把当前状态的价值往 target 方向拉一点。
  - 为什么这里没有更新动作？
    - 因为这是 state-value prediction，不是 control。
    - 这张图里动作 A 是：choose A according to fixed policy pi
      也就是说：动作已经由固定策略 π 决定了。
      算法不负责改策略，只负责评估：Vπ(s)
      也就是：如果一直按照策略 π 行动，那么每个状态值多少钱。
      所以表里面只需要存：V(s)
      而不需要存：Q(s,a)
    - 因为它不需要比较动作好坏。
      那动作的影响去哪了？
      动作虽然没有出现在 V(s) 表里，但它不是没有影响。
      动作的影响被策略 π 平均进去了。
      状态价值函数本质是：Vπ(s)=Eπ[Gt∣St=s]
      这里的 Eπ表示：在状态 s 下，动作是按照策略 π 选的。
      所以 Vπ(s) 评估的是：
      按照当前策略行动时，状态 s 的平均未来回报。
      如果你要改动作、学最优策略，那就不能只用 V(s)，而要用：
      Q(s,a)
      ~~这就是 SARSA、Q-learning 这些 control 方法要做的事。~~

$$Q(St, At) \leftarrow Q(St, At) + \alpha[Rt + \mathbf{1} + \gamma Q(St + \mathbf{1}, At + \mathbf{1}) - Q(St, At)]$$

- TD(0) 的动作价值版本上面的相当于还要手动加个policy improment 下面就不需要了
- Sarsa
  - 当前状态 S
  - 选当前动作 A
  - 执行动作 A
  - 得到奖励 R
  - 进入下一个状态 S_next
  - 在 S_next 下再选一个动作 A_next
    - 这是 SARSA 最关键的地方。
      它不是假设下一步一定选最优动作，而是说：下一步真实会按照当前策略 π 选动作。
      而当前策略是 ϵ-greedy，所以 Anext是从：ϵ-greedy(Q)里面真实采样出来的。
      因此 target 是：R+γQ(Snext,Anext)
      这表示：我这一步之后，下一步实际选择了 Anext，那我就用这个实际发生的动作价值来估计未来。
      这就是为什么 SARSA 是 on-policy。
    - 什么叫on-policy
      On-policy 的意思是：用来产生数据的策略，和被学习/评估的策略，是同一个策略。
      SARSA 里面：
      choose A using epsilon-greedy(Q)
      choose A_next using epsilon-greedy(Q)
      也就是说，智能体实际行动时用的是 ϵ-greedy 策略，更新 target 时也用这个 ϵ-greedy 策略真实选出来的动作。
      所以 SARSA 学到的是：qπ(s,a)其中 π 就是当前的 ϵ-greedy 策略。
  - 用 Q(S_next, A_next) 更新 Q(S, A)
    - 这个几乎所有一步一步和环境交互的算法都会有。
      因为智能体走了一步之后，当前位置变了：
      当前状态 S
      执行动作 A
      到达下一个状态 S_next
      下一轮循环时，智能体当然应该站在新的状态上继续行动，所以要写
      所以 TD(0)、SARSA、Q-learning 这类 online RL 算法都会有类似操作。
    - S = S_next 很常见
    - S = S_next
      A = A_next
      本轮用 A
      提前选好 A_next
      下一轮直接用 A_next，所以 SARSA 的动作是“接力式”的：
      - A = A_next 主要是 SARSA 这种需要“提前选好下一动作”的算法才明显需要
        因为 SARSA 是 on-policy。
        它学习的是：如果我真的按照当前 ϵ-greedy 策略行动，那么 Q(S,A) 应该是多少？
        所以它的 target 必须用真实策略实际选出来的下一动作：At+1~π(·∣St+1)
        - TD(0) 只有 S = S_next TD(0) 更新的是 V(s)，不关心动作价值，所以它不需要保存下一动作。
          Q-learning 通常也没有 A = A_next
          Q-learning 更新的是：Q(St,At)
          但是它的 target 是：Rt+1+γamaxQ(St+1,a)
  - Sarsa 的关键是：**A_next 真的来自当前 ϵ-greedy 策略**，所以它学的是当前策略自己的 $q_\pi(s,a)$。如果 Q 变了，下一次 ϵ-greedy 选动作的概率也会变，这就是边评估边改进。
- TD
  - Expected Sarsa
    - 仍然是 on-policy，但不用一个实际 A_next，而是对所有动作加权平均
      SARSA 是：
      这一次真实选到了哪个动作，我就用哪个动作。
      如果这次刚好探索到了一个很差的动作，target 就会比较低。
      Expected SARSA 是：
      我不看这一次真实抽到了哪个动作，我直接算当前策略整体的平均表现。
      所以 Expected SARSA 的 target 通常更稳定，方差更小。
  - n-step Sarsa
    - 前 n 步真实奖励，加上第 n 步后的 bootstrap。

```
initialize w

repeat for each transition:
    observe S, R, S_next under policy pi

    target = R + gamma * v_hat(S_next, w)
    delta  = target - v_hat(S, w)

    w = w + alpha * delta * grad_w v_hat(S, w)
```

- semi-gradient TD
  - semi-gradient Sarsa

> **把表格版 SARSA 的 $Q(S,A)$ 换成函数近似器 $\hat{q}(S,A,w)$，然后不再更新表格里的某一格，而是更新参数 w。**

也就是说，前面的表格 SARSA 是：

$$Q(S,A) \leftarrow Q(S,A) + \alpha\delta$$

现在变成：

$$w \leftarrow w + \alpha\delta\nabla_w \hat{q}(S,A,w)$$

核心更新是：

$$w \leftarrow w + \alpha\left[R + \gamma\hat{q}(S',A',w) - \hat{q}(S,A,w)\right]\nabla_w \hat{q}(S,A,w)$$

$$target = R + \gamma\hat{q}(S_{t+1}, A_{t+1}, w)$$

这和 SARSA 一样，仍然使用真实下一动作：

$$A_{t+1}$$

所以它还是 **on-policy**。

$$\delta = R + \gamma\hat{q}(S_{t+1}, A_{t+1}, w) - \hat{q}(S_t, A_t, w)$$

这就是 **SARSA TD error**。

它表示：

> 当前网络对 (S,A) 的估计，和 SARSA target 差了多少。

如果 $\delta > 0$，说明当前 $\hat{q}(S,A,w)$ 估低了；
如果 $\delta < 0$，说明当前 $\hat{q}(S,A,w)$ 估高了。

5. 为什么叫 semi-gradient?

这是最关键的地方。

你看 target 里面也有 w：

$$target = R + \gamma\hat{q}(S_{t+1}, A_{t+1}, w)$$

所以从数学上说，如果你对整个误差求完整梯度，target 这一项也应该对 w 求导。

但 semi-gradient SARSA **不对 target 求梯度。**

它只对当前预测项：

$$\hat{q}(S_t, A_t, w)$$

求梯度。

也就是说，它把：

$$R + \gamma\hat{q}(S_{t+1}, A_{t+1}, w)$$

暂时当成一个常数目标。

所以叫：

> **semi-gradient，半梯度。**

“半”的意思是：

> 只对被更新的当前预测 $\hat{q}(S,A,w)$ 求梯度，不对 target 里的 $\hat{q}(S',A',w)$ 求梯度。

6. 为什么可以这样做?

因为 TD 学习本来就是 bootstrapping：

$$\text{当前估计} \leftarrow \text{当前奖励} + \text{下一状态估计}$$

它不是标准监督学习里有固定真实标签，而是用当前估计构造临时目标。

如果你对 target 也求梯度，会让优化目标变得更复杂，而且不一定稳定。

所以经典 TD function approximation 通常使用 semi-gradient：

```
先计算一个 TD target
把这个 target 暂时看成常数
只更新当前 q_hat(S,A,w)
```

```
initialize w

repeat for each episode:
    initialize S
    choose A using epsilon-greedy(q_hat(.,.,w))

    while S is not terminal:
        take action A
        observe R, S_next
        choose A_next using epsilon-greedy(q_hat(.,.,w))

        target = R + gamma * q_hat(S_next, A_next, w)
        delta  = target - q_hat(S, A, w)

        w = w + alpha * delta * grad_w q_hat(S, A, w)

        S = S_next
        A = A_next
```